

DOCS

# Phase 1: Data Models & OOP Architecture

**Objective:** To define the structural "blueprints" for the hospital system using Python Classes. This phase focuses entirely on logic, data structure, and relationships between entities.

## 1.1 Class Hierarchy Overview

We will use **Multi-level Inheritance** to organize our data. This reduces code duplication (e.g., we don't need to write `name` and `age` for every single type of person).

- **Parent Class:** `Person` (Base entity)
- **Child Classes:** `Patient`, `Staff`
- **Grandchild Class:** `Doctor` (Inherits from `Staff`)
- **Standalone Managers:** `BloodInventory`, `MedicalAI`

## 1.2 Class Specifications

### A. Base Class: `Person`

- **Purpose:** Stores attributes common to every human in the system.
- **Attributes:**
  - `id` (String: Unique Identifier, e.g., "P-101", "D-505")
  - `name` (String)
  - `age` (Integer)
  - `contact_info` (String: Phone/Email)
- **Methods:**
  - `__init__`: Constructor to initialize basic details.
  - `get_details()`: Returns a dictionary of basic info.

### B. Child Class: `Patient` (Inherits `Person`)

- **Purpose:** Manages patient-specific medical data.
- **Additional Attributes:**
  - `blood_group` (String: e.g., "A+", "O-")
  - `medical_history` (List of Strings: e.g., ["Diabetes", "2023-Flu"])
  - `current_status` (String: "ADMITTED", "DISCHARGED", "PENDING")
  - `assigned_doctor_id` (String or None)
- **Key Methods:**
  - `admit()`: Sets status to "ADMITTED".
  - `discharge()`: Sets status to "DISCHARGED".

- `add_medical_record(record)`: Appends new disease/diagnosis to history.

### C. Child Class: **Staff** (Inherits **Person**)

- **Purpose:** General staff management.
- **Additional Attributes:**
  - `role` (String: "Nurse", "Admin", "Receptionist")
  - `shift_timing` (String: "Morning", "Night")

### D. Grandchild Class: **Doctor** (Inherits **Staff**)

- **Purpose:** specialized logic for doctors (appointments/predictions).
- **Additional Attributes:**
  - `specialization` (String: "Cardiology", "General")
  - `availability_slots` (List of Strings: e.g., ["10:00 AM", "11:00 AM"])
- **Key Methods:**
  - `assign_patient(patient_id)`: Links a patient to this doctor.
  - `remove_slot(time)`: Removes a slot once booked.

### E. Standalone Class: **BloodInventory**

- **Purpose:** Manages blood stock using NumPy (for Syllabus req).
- **Attributes:**
  - `blood_types` (List: ["A+", "A-", "B+", "B-", ...])
  - `stock_levels` (NumPy Array: [10, 5, 8, 2, ...])
  - `threshold_limit` (Integer: e.g., 5 units)
- **Key Methods:**
  - `update_stock(blood_type, quantity)`: Adds/subtracts stock.
  - `check_low_stock()`: Uses NumPy filtering to return types below threshold.
  - `get_stock_dataframe()`: Returns data formatted for Streamlit charts.

---

## 1.3 Python Implementation Preview (Skeletal Code)

This code belongs in your `models.py` file.

Python

```
import numpy as np
import datetime
```

```
# --- 1. Base Class ---
class Person:
```

```

def __init__(self, pid, name, age, contact):
    self.pid = pid
    self.name = name
    self.age = age
    self.contact = contact

def to_dict(self):
    """Helper to convert object to dict for saving to JSON"""
    return self.__dict__

```

# --- 2. Patient Class (Inheritance) ---

```

class Patient(Person):
    def __init__(self, pid, name, age, contact, blood_group):
        super().__init__(pid, name, age, contact) # Initialize parent
        self.blood_group = blood_group
        self.medical_history = []
        self.current_status = "PENDING"
        self.assigned_doctor_id = None

    def add_history(self, disease, prediction_notes=""):
        entry = {
            "date": str(datetime.date.today()),
            "disease": disease,
            "ai_prediction": prediction_notes
        }
        self.medical_history.append(entry)

```

# --- 3. Doctor Class (Multi-level Inheritance) ---

```

class Staff(Person):
    def __init__(self, pid, name, age, contact, role):
        super().__init__(pid, name, age, contact)
        self.role = role

class Doctor(Staff):
    def __init__(self, pid, name, age, contact, specialization):
        super().__init__(pid, name, age, contact, "Doctor")
        self.specialization = specialization
        self.available_slots = ["10:00-11:00", "11:00-12:00", "14:00-15:00"]

```

# --- 4. Inventory Manager (NumPy Integration) ---

```

class BloodInventory:
    def __init__(self):
        self.types = ["A+", "A-", "B+", "B-", "O+", "O-", "AB+", "AB-"]
        # Initialize with dummy data for POC

```

```
self.quantities = np.array([10, 5, 12, 4, 20, 8, 6, 2])
self.limit = 5

def get_low_stock(self):
    # NumPy Boolean Indexing (Syllabus: Advanced NumPy)
    low_indices = self.quantities < self.limit
    # Compress filters the list based on the boolean array
    return list(np.compress(low_indices, self.types))

def update_stock(self, blood_type, qty_change):
    if blood_type in self.types:
        idx = self.types.index(blood_type)
        self.quantities[idx] += qty_change
```

---

## 1.4 Next Step: Data Persistence

Now that we have the classes, the objects will disappear when you stop the script unless we save them.

# Phase 2: Backend Persistence – File Handling Engine

**Objective:** To implement a robust system that saves **Patient**, **Staff**, and **Inventory** data to JSON files so that data is not lost when the application is restarted. We will use **JSON** because it handles nested data (like a list of medical history records inside a patient object) much better than CSV.

## 2.1 File System Structure

The application will automatically check for and create a **data/** directory.

- **data/patients.json**: Stores a list of patient objects.
- **data/staff.json**: Stores a list of doctors and staff.
- **data/inventory.json**: Stores the current blood bank quantities.

**Example JSON Structure (patients.json):**

JSON

```
[
  {
    "pid": "P-101",
    "name": "John Doe",
    "medical_history": [
      {"date": "2023-10-01", "disease": "Flu", "ai_prediction": "Rest"}
    ]
  },
  {
    "pid": "P-102",
    "name": "Jane Smith",
    ...
  }
]
```

## 2.2 The Data Manager Module (`data_manager.py`)

This module serves as the "Interface" between your classes and the hard drive. It handles all the dirty work of opening, reading, and writing files.

### Key Functions:

1. `initialize_files()`:
  - *Logic*: Check if `data/` folder exists. If not, make it. Check if `patients.json` exists. If not, create an empty list `[]` inside it.
  - *Syllabus*: Modules, Directories, Exception Handling.
2. `load_data(filename)`:
  - *Logic*: Open the file in 'read' mode (`r`), parse JSON to a Python list of dictionaries, and return it.
  - *Error Handling*: If the file is corrupt or missing, return an empty list instead of crashing the app.
3. `save_data(filename, data_list)`:
  - *Logic*: Receive a list of Python dictionaries. Open file in 'write' mode (`w`). Dump the list to JSON.

---

## 2.3 Integration: Serialization & Deserialization

This is the most critical concept. You cannot save a Python **Object** directly to text. You must convert it.

### 1. **Serialization (Object → Dict):**

- We added `to_dict()` in Phase 1. Before saving, we loop through our list of Patient objects and call `p.to_dict()` on each.

### 2. **Deserialization (Dict → Object):**

- When loading, we get a list of Dictionaries. We need a helper method to convert them *back* into `Patient` objects so we can use methods like `.admit()` or `.add_history()`.

## 2.4 Implementation Snippet

Create a file named `data_manager.py`:

Python

import json

import os

DATA\_DIR = "data"

PATIENT\_FILE = os.path.join(DATA\_DIR, "patients.json")

STAFF\_FILE = os.path.join(DATA\_DIR, "staff.json")

INVENTORY\_FILE = os.path.join(DATA\_DIR, "inventory.json")

def ensure\_data\_dir():

"""Checks if data directory and files exist; creates them if not."""

if not os.path.exists(DATA\_DIR):

os.makedirs(DATA\_DIR)

# Initialize empty files if they don't exist

for filepath in [PATIENT\_FILE, STAFF\_FILE, INVENTORY\_FILE]:

if not os.path.exists(filepath):

with open(filepath, 'w') as f:

json.dump([], f) # Start with an empty list

def load\_json(filepath):

"""Generic function to read data from JSON."""

ensure\_data\_dir() # Safety check

try:

with open(filepath, 'r') as f:

return json.load(f)

except (FileNotFoundError, json.JSONDecodeError):

return [] # Return empty list on error (Exception Handling)

def save\_json(filepath, data):

"""Generic function to save data to JSON."""

ensure\_data\_dir()

```

try:
    with open(filepath, 'w') as f:
        json.dump(data, f, indent=4)
    return True
except Exception as e:
    print(f"Error saving data: {e}")
    return False

```

## 2.5 Updating `models.py` for Deserialization

Add this **Factory Method** to your `Patient` class in `models.py` so it can reconstruct itself from the loaded data.

Python

# Inside class Patient...

```

@classmethod
def from_dict(cls, data):
    """
    Reconstructs a Patient object from a dictionary.
    This is crucial for loading data back from JSON.
    """
    # Create new instance
    patient = cls(
        pid=data["pid"],
        name=data["name"],
        age=data["age"],
        contact=data["contact"],
        blood_group=data.get("blood_group", "Unknown")
    )
    # Restore complex attributes
    patient.medical_history = data.get("medical_history", [])
    patient.current_status = data.get("current_status", "PENDING")
    patient.assigned_doctor_id = data.get("assigned_doctor_id", None)

    return patient

```

---

## Phase 2 Summary Checklist

- [ ] `data_manager.py` created.
- [ ] Logic to create missing `data/` folder automatically is active.



- [] `Patient` class has `to_dict()` (from Phase 1) and `from_dict()` (new).

## Phase 3 (Updated): Backend Logic – Intelligence & Computation

**Objective:** To implement the computational logic (NumPy for Inventory) and integrate **Google Gemini 1.5 Flash** to analyze symptoms, predict resource usage, and suggest prescriptions while maintaining a "Human-in-the-loop" safety protocol.

### 3.1 Architecture of Logic Layer

- **Inventory Logic:** Remains the same (NumPy-based).
- **AI Engine:** Now uses `google-generativeai` library.
- **Safety Layer:** A mandatory "Disclaimer Wrapper" that appends a warning that all AI advice must be verified by the logged-in doctor.

### 3.2 Module B: The AI Doctor (Gemini 1.5 Flash Integration)

- **File:** `logic/ai_engine.py`
- **Syllabus Covered:** Modules, Exception Handling, Working with External APIs.
- **Prerequisite:** You will need to install the library: `pip install google-generativeai`

#### Key Features & Logic:

1. **Model Configuration:** specifically calls `gemini-1.5-flash` (chosen for speed and low latency, perfect for a realtime app).
2. **Structured Prompting:** The system prompt will force Gemini to output in a specific format (Diagnosis, Prescription, Resources).
3. **The "Doctor's Validation" Protocol:** The AI output is treated as a *draft*. The system stores it as "Suggested Treatment", which the doctor must explicitly "Confirm" in the UI later.

#### Implementation Snippet (`logic/ai_engine.py`)

Python

```
import google.generativeai as genai
import os
import json
```

```
class MedicalAI:
    def __init__(self):
```

```

# In a real app, use environment variables for security
# os.environ["GOOGLE_API_KEY"] = "YOUR_ACTUAL_API_KEY"

# Configure the library
try:
    genai.configure(api_key="YOUR_GEMINI_API_KEY_HERE")
    self.model = genai.GenerativeModel('gemini-1.5-flash')
    self.available = True
except Exception as e:
    print(f"Warning: AI Module failed to initialize. {e}")
    self.available = False

def predict_treatment(self, symptoms, patient_history):
    """
    Sends patient data to Gemini 1.5 Flash and retrieves
    structured advice including prescriptions.
    """

    if not self.available:
        return self._fallback_response()

    # 1. Construct the Prompt
    # We ask for JSON format to make it easy to display in Streamlit
    prompt = f"""
    Act as a senior medical assistant.
    Patient Symptoms: {symptoms}
    Patient Medical History: {patient_history}

    Task:
    1. Predict the likely diagnosis.
    2. Suggest a treatment plan.
    3. Suggest specific medicines (generic names) with dosage.
    4. List hospital resources needed (e.g., Oxygen, ICU, General Ward).

    CRITICAL CONSTRAINT: Output ONLY valid JSON in the following format:
    {{
        "diagnosis": "string",
        "treatment_plan": "string",
        "suggested_rx": ["medicine 1", "medicine 2"],
        "resources": ["resource 1", "resource 2"],
        "risk_level": "Low/Medium/High"
    }}
    """

    try:

```

```

# 2. Call Gemini API
response = self.model.generate_content(prompt)

# 3. Parse Response (Basic cleaning to ensure valid JSON)
raw_text = response.text.strip()
# Remove any markdown formatting if Gemini adds it (like ```json ... ```)
if raw_text.startswith("```"):
    raw_text = raw_text.strip("`").replace("json", "").strip()

ai_data = json.loads(raw_text)

# 4. Attach Safety Disclaimer (The "Doctor's Decision" Rule)
ai_data["disclaimer"] = "⚠️ AI SUGGESTION ONLY. Doctor must verify before approval."

return ai_data

except Exception as e:
    # Syllabus: Exception Handling
    return {
        "diagnosis": "Error in AI Analysis",
        "treatment_plan": f"Manual Check Required. System Error: {str(e)}",
        "suggested_rx": [],
        "resources": [],
        "disclaimer": "System Failure"
    }

def _fallback_response(self):
    """Used if API key is missing or internet is down"""
    return {
        "diagnosis": "System Offline",
        "treatment_plan": "AI Service Unavailable. Please proceed with manual diagnosis.",
        "suggested_rx": ["Manual Entry Required"],
        "resources": [],
        "disclaimer": "Offline Mode"
    }

```

### 3.3 Integration Plan for Phase 4 (UI)

When we build the Frontend next, we will handle the "Doctor's Decision" logic as follows:

1. **Display:** Show the AI's JSON output in a nice card format.
2. **Edit:** Provide a text box pre-filled with the AI's **suggested\_rx**.

3. **Action:** The Doctor can *edit* the prescription (e.g., change dosage) and then click a **"Finalize & Prescribe"** button. This saves the *edited* version to the patient's file, not the raw AI output.
- 

### Phase 3 (Updated) Checklist

- [ ] `google-generativeai` library added to requirements.
- [ ] `MedicalAI` class updated to use `gemini-1.5-flash`.
- [ ] Prompt engineering ensures JSON output for easy parsing.
- [ ] Safety Disclaimer added to every AI response.

## Phase 4: Frontend Integration – Streamlit UI

**Objective:** To build an interactive, role-based Web Interface that connects the user to the Backend Logic (Phase 1-3). The UI will demonstrate the "Visualization" and "Streamlit UI" syllabus requirements.

### 4.1 UI Architecture & Navigation

We will use `st.sidebar` to simulate a "Role-Based Access Control" system. This keeps the interface clean and separates the concerns of a Receptionist vs. a Doctor.

#### Navigation Options (Sidebar):

1. 🏠 **Reception Desk:** For Patient Registration & Administration.
  2. 👨 **Doctor's Cabin:** For AI Consultation & Treatment.
  3. 🩸 **Blood Bank:** For Inventory Management & Visualization.
- 

### 4.2 Module A: Reception Desk (CRUD UI)

- **Goal:** Manage Patient Records.
- **Syllabus:** Input Widgets, DataFrames, Session State.

#### Features:

- **Registration Form:** `st.text_input` fields for Name, Age, Contact.
- **Patient List:** Display the list of patients using `st.dataframe()`.
- **Discharge Button:** A button next to a patient's name to change their status from `ADMITTED` to `DISCHARGED`.

### 4.3 Module B: Doctor's Cabin (The AI Interface)

- **Goal:** Diagnose patients using Gemini 1.5 Flash.
- **Syllabus:** Text Areas, JSON Display, Button Callbacks.

#### Workflow:

1. **Select Patient:** Dropdown menu (`st.selectbox`) to choose a patient from the registered list.
2. **Symptom Input:** Large text area (`st.text_area`) for the doctor to type symptoms/observations.
3. **AI Consultation:** A button "Consult AI" triggers the `MedicalAI.predict_treatment()` method.
4. **Review & Prescribe:**
  - **Left Column:** Displays AI suggestions (Diagnosis, Rx) in a read-only JSON/Card view.
  - **Right Column:** Editable text area pre-filled with AI's suggestion. The doctor edits this and clicks "Confirm Treatment" to save it to the patient's history.

### 4.4 Module C: Blood Bank (Visualization)

- **Goal:** Visual analytics of blood stock.
- **Syllabus:** Advanced NumPy, Streamlit Charts (`st.bar_chart`).

#### Features:

- **Update Stock:** Numeric inputs to Add/Remove blood bags.
- **Visuals:** A Bar Chart showing quantities of all blood types.
- **Alerts:** If the Backend `check_critical_levels()` returns data, display a large `st.error("CRITICAL STOCK: A+, B-")` banner at the top.

### 4.5 Implementation Snippet (`main.py`)

This is the entry point of your application.

```
Python
import streamlit as st
```

```

import pandas as pd
import numpy as np
from data_manager import load_json, save_json
from models import Patient, BloodInventory
from logic.ai_engine import MedicalAI

# --- CONFIGURATION ---
st.set_page_config(page_title="Smart Hospital POC", layout="wide")
ai_engine = MedicalAI()

# --- SIDEBAR NAVIGATION ---
st.sidebar.title("🏥 Hospital System")
menu = st.sidebar.radio("Go to:", ["Reception Desk", "Doctor's Cabin", "Blood Bank"])

# =====
# 1. RECEPTION DESK (CRUD)
# =====

if menu == "Reception Desk":
    st.title("Patient Registration & Management")

    # Form to Add Patient
    with st.expander("✚ Register New Patient"):
        with st.form("add_patient_form"):
            name = st.text_input("Full Name")
            age = st.number_input("Age", min_value=0, max_value=120)
            contact = st.text_input("Contact Number")
            b_group = st.selectbox("Blood Group", ["A+", "A-", "B+", "B-", "O+", "O-", "AB+", "AB-"])

            submitted = st.form_submit_button("Register Patient")
            if submitted:
                # Logic to create Patient Object and Save to JSON
                # p = Patient(...)
                # save_new_patient(p)
                st.success(f"Patient {name} Registered Successfully!")

    # View Patients
    st.subheader("Current Patient List")
    # Load data (Mocking data loading for snippet)
    patients_data = load_json("data/patients.json")
    if patients_data:
        df = pd.DataFrame(patients_data)
        st.dataframe(df[["pid", "name", "blood_group", "current_status"]])
    else:
        st.info("No patients registered yet.")

```

```

# =====
# 2. DOCTOR'S CABIN (AI INTEGRATION)
# =====
elif menu == "Doctor's Cabin":
    st.title("👨‍⚕️ AI-Assisted Diagnosis")

    # Select Patient
    patients = load_json("data/patients.json")
    patient_names = [p['name'] for p in patients]
    selected_name = st.selectbox("Select Patient", patient_names)

    col1, col2 = st.columns(2)

    with col1:
        st.subheader("🩺 Assessment")
        symptoms = st.text_area("Enter Symptoms & Observations", height=150)

        if st.button("Consult AI Assistant"):
            with st.spinner("Analyzing with Gemini 1.5 Flash..."):
                # Call the AI Engine
                # We fetch history from the selected patient object
                history = "Diabetic, Hypertension" # Mock history

                prediction = ai_engine.predict_treatment(symptoms, history)

                # Store in Session State so it doesn't vanish on reload
                st.session_state['ai_result'] = prediction

    with col2:
        st.subheader("💊 Treatment Plan")
        if 'ai_result' in st.session_state:
            res = st.session_state['ai_result']

            # Display AI Warning
            st.warning(res.get("disclaimer", "Verify Output"))

            # Display Diagnosis
            st.info(f"***Diagnosis:** {res.get('diagnosis')}")

            # Editable Prescription (Human-in-the-loop)
            rx_text = "\n".join(res.get('suggested_rx', []))
            final_rx = st.text_area("Prescription (Doctor's Edit)", value=rx_text)

```

```

        if st.button("✅ Finalize & Save to History"):
            st.success("Treatment Plan Saved to Patient Record.")
            # Logic to append to patient.medical_history goes here

# =====
# 3. BLOOD BANK (NUMPY VISUALIZATION)
# =====
elif menu == "Blood Bank":
    st.title("🩸 Blood Inventory Management")

    # Initialize Logic
    inventory = BloodInventory() # From Phase 1/3

    # Check Alerts (Advanced NumPy Logic)
    low_stock_items = inventory.get_low_stock()
    if low_stock_items:
        st.error(f"⚠️ URGENT ALERT: Low stock for {', '.join(low_stock_items)}")

    # Visualization
    col1, col2 = st.columns([2, 1])

    with col1:
        st.subheader("Stock Levels")
        # Convert Inventory Object data to DataFrame for Chart
        chart_data = pd.DataFrame({
            "Blood Type": inventory.types,
            "Units": inventory.quantities
        }).set_index("Blood Type")

        st.bar_chart(chart_data)

    with col2:
        st.subheader("Update Inventory")
        b_type = st.selectbox("Type", inventory.types)
        qty = st.number_input("Units to Add/Remove", step=1)
        if st.button("Update Stock"):
            # inventory.update_stock(b_type, qty)
            st.success("Inventory Updated")

```

---

## Phase 4 Summary Checklist

- [ ] `main.py` created with Sidebar Navigation.



- [ ] Doctor's module successfully connects to `ai_engine.py`.
- [ ] Human-in-the-loop workflow (AI suggests → Doctor Edits → Save) is implemented.
- [ ] Blood Bank visualizes NumPy arrays using `st.bar_chart`.

## Phase 5: The Gatekeeper (RBAC & Admin)

- **Goal:** Secure the app and manage staff.
- **New File:** `auth_manager.py` (Handles login verification).
- **UI Changes:**
  - Create a **Login Page** that hides the main app until authenticated.
  - Create an **Admin Dashboard** (only visible to Admin) to register Doctors/Staff into `staff.json`.

## Phase 6: The Appointment Engine

- **Goal:** Connect Patients to Doctors via Time Slots.
- **New File:** `appointments.json` (Stores: `appointment_id`, `patient_id`, `doctor_id`, `time`, `status`).
- **UI Changes:**
  - **Staff:** Add "Schedule Appointment" tab.
  - **Doctor:** Add "My Calendar" tab (Filters appointments by logged-in Doctor ID).

## Phase 7: The Clinical Loop (Fixing History & AI)

- **Goal:** Make the "Doctor's Cabin" fully functional.
- **Logic Fixes:**
  - **Save History:** Replace the mock comment with actual `save_json` logic.
  - **AI Context:** Ensure the `history` passed to Gemini includes the *previous* appointments, not just the current session.
  - **Editable Rx:** Implement the "Human-in-the-loop" text area we discussed earlier.

I am fixing a bug in my Smart Hospital System where the 'Patient History' in the 'Doctor's Cabin' shows 'Unknown' or 'N/A' even after saving.

Requirements for the generated code:

1. Update 'main.py' - Doctor's Cabin History Expander:

- Ensure the loop through 'medical\_history' uses the keys 'diagnosis' (instead of 'disease') and 'treatment' (instead of 'details') to match my JSON structure.
- Include the 'date' and 'doctor\_id' in the history display for accountability.

2. Update 'main.py' - Finalize & Save Button Logic:

- When the 'Finalize & Save Treatment' button is clicked, it must create a dictionary with keys: 'date', 'diagnosis', 'treatment', and 'doctor\_id'.
- The 'diagnosis' value should come from the AI's 'diagnosis' field, and 'treatment' should come from the 'final\_notes' text area.
- Use 'load\_json' to get the latest patient data, append the new record to the specific patient's 'medical\_history' list, and use 'save\_json' to commit changes to 'patients.json'.
- Ensure 'st.rerun()' is called immediately after saving to refresh the UI and show the new history entry.

### 3. Logic Consistency:

- Ensure the AI response parsing logic in 'MedicalAI' matches these keys so that the 'final\_notes' text area is pre-populated correctly.

Constraint: Maintain the current session state management for the logged-in doctor to ensure 'doctor\_id' is captured accurately from 'st.session\_state[user]'.

# Prompts

## Phase 1: Backend Core – Data Models & OOP

**Goal:** Build the logical "skeleton" of the application using Python Classes. **Syllabus Target:** Intro to OOP, Advanced OOP (Inheritance), Mutable Data Structures, Modules.

### Sub-Phase 1.1: The Base **Person** Class

- **Focus:** Creating the parent class to handle shared attributes (ID, Name, Contact).
- **Concepts:** Class definition, Constructor (`__init__`), Object attributes.

#### Prompt for Antigravity (Copy & Paste):

Plaintext

I am building a Hospital Management System in Python.  
I need you to create a file named `models.py`.

Step 1: Create a base class named `Person`.

- This class should have an `__init__` method that accepts: `pid` (ID), `name`, `age`, and `contact`.
- Store these as instance attributes.
- Add a method called `to_dict(self)` that returns a Python dictionary representation of the object (using `self.__dict__`).
- This method is critical for later saving data to JSON.

Constraint:

- Use standard Python class syntax.
  - Do not use dataclasses or Pydantic (strictly follow "Introduction to Object Oriented Programming" syllabus).
- 

### Sub-Phase 1.2: The **Patient** Class (Inheritance)

- **Focus:** Inheriting from **Person** and adding medical-specific data.
- **Concepts:** Inheritance (`super()`), Mutable Lists (Medical History).

#### Prompt for Antigravity (Copy & Paste):

Plaintext

Now, extend `models.py` by adding a `Patient` class.

Requirements:

1. Inheritance: The `Patient` class must inherit from the `Person` class.
2. Constructor: It should accept `pid`, `name`, `age`, `contact`, and `blood\_group`.

- Use ``super().__init__(...)`` to initialize the parent attributes.

### 3. Attributes:

- ``self.blood_group``: String.
- ``self.medical_history``: An empty list `[]`` (Mutable Data Structure).
- ``self.current_status``: String, default is "PENDING".
- ``self.assigned_doctor_id``: None by default.

### 4. Methods:

- ``add_history(self, disease, prediction_notes)``: This creates a dictionary with keys "date" (use `datetime.date.today()`), "disease", and "ai\_prediction", and appends it to ``self.medical_history``.
- ``admit(self)``: Sets ``current_status`` to "ADMITTED".
- ``discharge(self)``: Sets ``current_status`` to "DISCHARGED".

Constraint: Ensure you import ``datetime`` at the top of the file.

---

## Sub-Phase 1.3: The **Staff & Doctor** Hierarchy (Multi-level Inheritance)

- **Focus:** A 3-level hierarchy to satisfy "Advanced OOP".
- **Concepts:** Multi-level Inheritance, Method Overriding/Extension.

### Prompt for Antigravity (Copy & Paste):

Plaintext

We need to handle hospital staff using Multi-level Inheritance in ``models.py``.

Step 1: Create a ``Staff`` class that inherits from ``Person``.

- Add an attribute ``role`` (e.g., "Nurse", "Admin") and ``shift_timing``.
- Initialize these in ``__init__`` along with the parent attributes.

Step 2: Create a ``Doctor`` class that inherits from ``Staff``.

- The ``role`` should be automatically set to "Doctor" inside the constructor.
- Add specific attributes: ``specialization`` and ``available_slots`` (a list of strings like ["10-11", "11-12"]).
- Add a method ``assign_patient(self, patient_id)`` which links a patient ID to the doctor (you can just store it in a list or print a confirmation for now).

Constraint: This demonstrates the "Advanced OOP Concepts" from my syllabus. Ensure the inheritance chain is clear: Person -> Staff -> Doctor.

---

## Sub-Phase 1.4: The **BloodInventory** Class (NumPy Integration)

- **Focus:** Managing blood stock using NumPy arrays instead of standard lists.

- **Concepts:** NumPy Arrays, Boolean Indexing (Filtering).

### Prompt for Antigravity (Copy & Paste):

Plaintext

Finally, add a `BloodInventory` class to `models.py`. This class must use the `numpy` library strictly.

Requirements:

1. Attributes:

- `self.types`: A standard Python list ["A+", "A-", "B+", "B-", "O+", "O-", "AB+", "AB-"].
- `self.quantities`: A **NumPy array** of integers representing the stock level for each type (initialize with dummy values like `np.array([10, 5, ...])`).
- `self.limit`: An integer (e.g., 5) representing the low-stock threshold.

2. Methods:

- `get\_low\_stock(self)`: This **MUST** use NumPy boolean indexing (e.g., `self.quantities < self.limit`) to find and return a list of blood types that are low on stock. Do not use a for-loop for this check.
- `update\_stock(self, blood\_type, qty\_change)`: Find the index of the blood type and update the corresponding index in the NumPy array.

Constraint: Ensure `import numpy as np` is at the top. This class satisfies the "Introduction to NumPy" syllabus requirement.

## Phase 2: Backend Persistence – File Handling Engine

### Sub-Phase 2.1: The Data Manager Setup

- **Focus:** Creating the module and ensuring the file system is ready.
- **Concepts:** Modules, Directories (os module), File Paths.

### Prompt for Antigravity (Copy & Paste):

Plaintext

I need to implement file persistence for my Hospital System.  
Create a new file named `data\_manager.py`.

Step 1: Setup Constants and Directory Logic.

- Define a constant `DATA\_DIR = "data"`.
- Define file paths:
  - `PATIENT\_FILE = os.path.join(DATA\_DIR, "patients.json")`
  - `STAFF\_FILE = os.path.join(DATA\_DIR, "staff.json")`
  - `INVENTORY\_FILE = os.path.join(DATA\_DIR, "inventory.json")`

Step 2: Create a function `ensure_data_dir()`.

- Check if `DATA_DIR` exists using `os.path.exists`. If not, create it using `os.makedirs`.
- Loop through the three file paths defined above.
- If a file does not exist, create it and write an empty JSON list `[]` into it.
- Use the `json` module for this.

Constraint: This satisfies the "Modules, Directories" and "Working with Files" syllabus requirements.

---

### Sub-Phase 2.2: JSON Operations with Exception Handling

- **Focus:** Robust reading and writing of data.
- **Concepts:** Functions, JSON Serialization, Exception Handling (`try-except`).

#### Prompt for Antigravity (Copy & Paste):

Plaintext

Now, add the core file operations to `data_manager.py`.

Step 1: Create `load_json(filepath)`.

- Call `ensure_data_dir()` at the start to be safe.
- Use a `try-except` block:
  - Try to open the file in 'read' mode (`'r'`) and return `json.load(f)`.
  - Catch `FileNotFoundError` and `json.JSONDecodeError`. If these occur, return an empty list `[]`.

Step 2: Create `save_json(filepath, data)`.

- Call `ensure_data_dir()` at the start.
- Use a `try-except` block:
  - Open the file in 'write' mode (`'w'`).
  - Use `json.dump(data, f, indent=4)` to save the list.
  - Return `True` if successful, `False` if an exception occurs (and print the error).

Constraint: This demonstrates "Exception Handling" (`try/except`) effectively.

---

### Sub-Phase 2.3: Updating Models for Deserialization

- **Focus:** Converting raw Dictionaries (from JSON) back into Python Objects.
- **Concepts:** Class Methods (`@classmethod`), Dictionary manipulation.

### Prompt for Antigravity (Copy & Paste):

Plaintext

We need to update `models.py` to handle loading data back from JSON.

Task: Go to the `Patient` class in `models.py` and add a Factory Method.

Add this method specifically:

```
@classmethod
def from_dict(cls, data):
    """
    Reconstructs a Patient object from a dictionary.
    """
    # 1. Create the instance using the basic constructor arguments
    patient = cls(
        pid=data["pid"],
        name=data["name"],
        age=data["age"],
        contact=data["contact"],
        blood_group=data.get("blood_group", "Unknown")
    )

    # 2. Manually restore the complex/mutable attributes
    # Use .get() with defaults to prevent errors if keys are missing
    patient.medical_history = data.get("medical_history", [])
    patient.current_status = data.get("current_status", "PENDING")
    patient.assigned_doctor_id = data.get("assigned_doctor_id", None)

    return patient
```

Constraint: This allows us to convert the list of dicts we get from `data\_manager.py` back into fully functional `Patient` objects.

## Phase 3: Backend Logic – Intelligence & Computation

### Sub-Phase 3.1: Module Setup & Configuration

- **Focus:** Organizing the logic into a separate package and setting up the Gemini client.
- **Concepts:** Modules, Directories, API Configuration.

### Prompt for Antigravity (Copy & Paste):

Plaintext

I need to build the AI Logic module for the Hospital System.



Step 1: Create a new folder named `logic`.

- Inside `logic`, create an empty `\_\_init\_\_.py` file (to make it a Python package).
- Create a new file named `ai\_engine.py`.

Step 2: In `ai\_engine.py`, setup the `MedicalAI` class.

- Import `google.generativeai` as `genai`, `os`, and `json`.
- Define the class `MedicalAI`.
- In the `\_\_init\_\_` method:
  - Configure the API using `genai.configure(api\_key="YOUR\_API\_KEY")`. (Note: Leave a placeholder string for the key).
  - Initialize the model: `self.model = genai.GenerativeModel('gemini-1.5-flash')`.
  - Wrap this setup in a `try-except` block. If it fails (e.g., no internet or bad key), set a flag `self.available = False`. Else set it to `True`.

Constraint: This setup must be robust. If the API fails to load, the app should not crash.

---

### Sub-Phase 3.2: The Prediction Logic (Prompt Engineering)

- **Focus:** Getting structured JSON data from the LLM.
- **Concepts:** Prompt Engineering, JSON Parsing, Exception Handling.

#### Prompt for Antigravity (Copy & Paste):

Plaintext

Now, implement the core prediction method in `ai\_engine.py`.

Task: Add a method `predict\_treatment(self, symptoms, patient\_history)` to the `MedicalAI` class.

Logic Flow:

1. Check `if not self.available`: Return a fallback dictionary (dummy data) saying "System Offline".
2. Construct the Prompt:
  - Act as a senior medical assistant.
  - Input: Symptoms and Patient History.
  - Output Requirement: Strict JSON format only.
  - Fields required in JSON: "diagnosis", "treatment\_plan", "suggested\_rx" (list of meds), "resources" (list of hospital resources like Oxygen, Bed), "risk\_level".
3. Call the API: `response = self.model.generate\_content(prompt)`.
4. Parse the Response:
  - Clean the text (remove markdown backticks if present).
  - Use `json.loads()` to convert the text into a Python Dictionary.

5. Error Handling: Wrap the API call and parsing in a `try-except` block. Return a safe error dictionary if anything breaks.

Constraint: Ensure the prompt explicitly demands "ONLY valid JSON" to prevent parsing errors.

---

### Sub-Phase 3.3: The Safety Layer (Human-in-the-Loop)

- **Focus:** Ethical AI usage.
- **Concepts:** Dictionary Manipulation, Defensive Programming.

#### Prompt for Antigravity (Copy & Paste):

Plaintext

Finally, add the Safety Layer to `ai\_engine.py`.

Task: Modify the `predict\_treatment` method (or add a helper) to inject a disclaimer.

Requirement:

- Before returning the `ai\_data` dictionary from the `try` block, add a new key:  
`ai\_data["disclaimer"] = "⚠️ AI SUGGESTION ONLY. Doctor must verify before approval."`
- Create a private method `\_fallback\_response(self)` that returns a hardcoded dictionary with the same structure (diagnosis, treatment\_plan, etc.) but with values like "Service Unavailable" and "Manual Check Required". Use this in the `except` blocks.

Constraint: This ensures that even if the AI works perfectly, the "Human-in-the-loop" protocol is enforced via data structure.

## Phase 4: Frontend Integration – Streamlit UI

### Sub-Phase 4.1: App Skeleton & Sidebar Navigation

- **Focus:** Setting up the main entry point and the role-based navigation system.
- **Concepts:** Streamlit Setup, Sidebar Layout.

#### Prompt for Antigravity (Copy & Paste):

Plaintext

I am now building the Frontend. Create a file named `main.py`.

Step 1: Imports & Config.

- Import `streamlit as st`, `pandas as pd`, and `numpy as np`.

- Import our backend modules: ``from data_manager import load_json, save_json`, `from models import Patient, BloodInventory`, `from logic.ai_engine import MedicalAI`.`
- Set page config: ``st.set_page_config(page_title="Smart Hospital POC", layout="wide")``.

Step 2: Sidebar Navigation.

- Use ``st.sidebar.title("🏥 Hospital System")``.
- Create a radio button menu: ``menu = st.sidebar.radio("Go to:", ["Reception Desk", "Doctor's Cabin", "Blood Bank"])``.

Step 3: Skeleton Logic.

- Write simple ``if menu == "Reception Desk":`` blocks for each option.
- Just put ``st.title("Reception")`` (etc.) inside each block for now to test the navigation.

Constraint: This file links all previous modules together.

## Sub-Phase 4.2: Reception Desk (Patient CRUD UI)

- **Focus:** Data Entry and Display.
- **Concepts:** Input Widgets (`text_input`, `selectbox`), DataFrames, Lists.

### Prompt for Antigravity (Copy & Paste):

Plaintext

Now, implement the "Reception Desk" logic inside ``main.py``.

Step 1: Registration Form.

- Use ``with st.expander("✚ Register New Patient"):``.
- Create a form ``with st.form("add_patient"):``.
- Add widgets for Name (text), Age (number), Contact (text), and Blood Group (selectbox with options `["A+", "A-", ...]`).
- Add a Submit button.
- On submit:
  1. Create a new ``Patient`` object (import ``Patient`` class).
  2. Load current patients (``load_json``), append the new patient's ``.to_dict()``, and save back (``save_json``).
  3. Show ``st.success``.

Step 2: Display Patients.

- Outside the expander, load the patient list again.
- If the list is not empty, convert it to a Pandas DataFrame.
- Display it using ``st.dataframe(df[["pid", "name", "blood_group", "current_status"]])`` for a clean view.

Constraint: This covers "Input Widgets" and "Visualization" (Tables).

---

### Sub-Phase 4.3: Doctor's Cabin (AI UI & Session State)

- **Focus:** Interactive AI workflow.
- **Concepts:** Session State (crucial for keeping AI results visible), Column Layouts.

#### Prompt for Antigravity (Copy & Paste):

Plaintext

Implement the "Doctor's Cabin" logic in ``main.py``.

Step 1: Patient Selection.

- Load patient data. Create a selectbox ``selected_patient`` listing patient names.

Step 2: AI Interaction Layout.

- Create two columns: ``col1, col2 = st.columns(2)``.

- In ``col1`` (Assessment):

- Add ``st.text_area`` for "Symptoms".

- Add a button "Consult AI".

- When clicked: Call ``MedicalAI().predict_treatment()``.

- **\*\*CRITICAL:\*\*** Save the result into ``st.session_state['ai_result']`` so it doesn't disappear when the page reloads.

Step 3: Result Display & Confirmation.

- In ``col2`` (Treatment Plan):

- Check ``if 'ai_result' in st.session_state``.

- Display the result using ``st.json`` or ``st.info``.

- Show the "Disclaimer" in a ``st.warning``.

- Add a "Finalize Treatment" button (mock logic to save to history is fine).

Constraint: Explicitly use ``st.session_state`` to manage the AI response persistence.

---

### Sub-Phase 4.4: Blood Bank (NumPy Visualization)

- **Focus:** Visualizing the NumPy array from Phase 1.
- **Concepts:** Streamlit Charts (`bar_chart`), NumPy Integration.

#### Prompt for Antigravity (Copy & Paste):

Plaintext

Finally, implement the "Blood Bank" logic in ``main.py``.

Step 1: Initialize Inventory.

- Instantiate ``inventory = BloodInventory()``. (This class uses NumPy internally).

Step 2: Critical Alert.

- Call ``inventory.get_low_stock()``.
- If it returns a list, display ``st.error(f"⚠️ Low Stock: {list}")`` at the very top.

Step 3: Visualization.

- Create a Pandas DataFrame from the inventory data: ``pd.DataFrame({"Type": inventory.types, "Units": inventory.quantities})``.
- Set "Type" as the index.
- Display a Bar Chart: ``st.bar_chart(df)``.

Step 4: Update Logic.

- Add a simple UI (selectbox for Type, number\_input for Qty) to update stock.
- Add a button "Update" that calls ``inventory.update_stock()``.

Constraint: This satisfies the "Visualization" and "Introduction to NumPy" syllabus requirements.

## Phase 5: Auth & Admin

### 5.1: Update Models for Auth

We need to add `username` and `password` to the `Staff` class so they can log in.

#### Prompt for Antigravity:

Plaintext

I need to update ``models.py`` to support Authentication.

Task: Modify the ``Staff`` and ``Doctor`` classes.

1. Update ``Staff.__init__``: Add ``username`` and ``password`` parameters. Store them as attributes.
2. Update ``Doctor.__init__``: Pass ``username`` and ``password`` up to the ``Staff`` super constructor.
3. Update ``from_dict`` methods (if you have them) or ensure ``to_dict`` includes these new fields.
4. Add a standard method ``verify_password(self, input_password)`` to ``Staff`` that returns True if they match.

Constraint: Keep using standard Python classes.

## 5.2 Create the Auth Manager

We need a helper to check credentials against `staff.json`.

### Prompt for Antigravity:

Plaintext

Create a new file ``auth.py``.

Functionality:

1. ``login(username, password)``:

- Load ``staff.json``.
- specific check: If `username=="admin"` and `password=="admin123"`, return a mock Admin object (Role: "Admin").
- Else, loop through the staff list. If a match is found, return the Staff/Doctor object (as a dictionary).
- Return None if failed.

2. Session Management:

- We will handle session state in ``main.py``, so this file just handles the logic of verification.

## 5.3 Admin Dashboard (Manage Staff)

This allows us to actually create users who can log in.

### Prompt for Antigravity:

Plaintext

In ``main.py``, we need to implement the Admin View.

1. Create a function ``admin_dashboard()``:

- Form: "Add New Staff/Doctor".
- Inputs: Name, Role (Doctor/Nurse/Receptionist), Specialization (if Doctor), Username, Password.
- Action: Create the ``Staff`` or ``Doctor`` object, save to ``staff.json`` using ``save_json``.
- Display: A table of current staff (excluding passwords).

Constraint: Only show this if ``st.session_state['role'] == 'Admin'``.

## 5.4: The Login Screen (Refactoring Main)

We need to hide the entire app behind a login wall.

## Prompt for Antigravity:

Plaintext

Refactor ``main.py`` to implement a Login Flow.

1. Check ``if 'user' not in st.session_state``:
  - Show a Login Form (Username/Password).
  - On Submit: Call ``auth.login()``.
  - If success: Set ``st.session_state['user']`` and ``st.session_state['role']``. Rerun app.
  - If fail: Show error.
2. If user IS logged in:
  - Show the Sidebar.
  - **\*\*Dynamic Sidebar:\*\***
    - If Role == "Admin": Show ["Admin Dashboard"].
    - If Role == "Doctor": Show ["Doctor's Cabin"].
    - If Role == "Receptionist": Show ["Reception Desk", "Appointments"].
  - Add a "Logout" button in the sidebar that clears session state.

## Sub-Phase 5.1: Updating Models for Auth

- **Focus:** Adding credentials to the Staff classes.
- **Concepts:** Inheritance, Method Overriding.

## Prompt for Antigravity (Copy & Paste):

Plaintext

I need to add Authentication to my Hospital System.

Update ``models.py`` to support login credentials.

Task: Modify the ``Staff`` and ``Doctor`` classes.

1. Update ``Staff.__init__``:
  - Add ``username`` and ``password`` parameters to the constructor.
  - Store them as ``self.username`` and ``self.password``.
  - Add a method ``verify_password(self, input_password)`` that returns ``True`` if ``input_password == self.password``.
2. Update ``Doctor.__init__``:
  - Accept ``username`` and ``password`` in the arguments.
  - Pass them to the ``super().__init__`` call so the parent ``Staff`` class handles them.
3. Update ``from_dict`` / ``to_dict``:

- Ensure these new fields are saved and loaded correctly from JSON.

Constraint: Keep using standard Python classes. Do not use external auth libraries yet; we are building this logic manually for the POC.

### Sub-Phase 5.2: The Auth Manager Module

- **Focus:** Logic to verify users against the database.
- **Concepts:** Modules, File I/O, Logic.

#### Prompt for Antigravity (Copy & Paste):

Plaintext

Create a new file named `auth\_manager.py`.

Functionality:

1. Import `load\_json` from `data\_manager`.
2. Define a constant `ADMIN\_CREDENTIALS = {"username": "admin", "password": "admin123"}`.
3. Create a function `login\_user(username, password)`:
  - **\*\*Admin Check:\*\*** First, check if the input matches `ADMIN\_CREDENTIALS`. If yes, return a dictionary: `{"name": "Administrator", "role": "Admin", "username": "admin"}`.
  - **\*\*Staff Check:\*\*** If not admin, load the staff list using `load\_json("data/staff.json")`.
  - Loop through the list. If `user['username'] == username` AND `user['password'] == password`, return that user dictionary.
  - If no match found, return `None`.

Constraint: This keeps the authentication logic separate from the UI.

### Sub-Phase 5.3: Refactoring Main for Login & Admin

- **Focus:** Hiding the app behind a login wall and adding the Admin Panel.
- **Concepts:** Session State Management, Conditional UI.

#### Prompt for Antigravity (Copy & Paste):

Plaintext

We need to refactor `main.py` to add a Login Screen and Admin capabilities.

Step 1: Session State Initialization

- At the top of `main.py`, check: `if 'logged\_in' not in st.session\_state: st.session\_state['logged\_in'] = False`.



### Step 2: Login Logic (The "Gatekeeper")

- If ``st.session_state['logged_in']`` is False:
  - Show a ``st.form`` with Username and Password.
  - On submit, call ``auth_manager.login_user()``.
  - If successful, set ``st.session_state['logged_in'] = True``, save the ``user`` object and ``role`` in session state, and ``st.rerun()``.
  - If failed, show ``st.error("Invalid Credentials")``.

### Step 3: The Admin Dashboard

- If ``st.session_state['role'] == "Admin"``:
  - Add a new Sidebar Option: "Admin Dashboard".
  - In this dashboard, create a form to **\*\*Add New Staff\*\***.
  - Inputs: Name, Role (Doctor/Nurse/Receptionist), Specialization (optional), Username, Password.
  - Logic: Create a ``Staff`` or ``Doctor`` object, convert ``to_dict()``, append to ``staff.json``, and save.

### Step 4: Logout

- Add a "Logout" button in the sidebar. If clicked, clear session state and rerun.

Constraint: Ensure the existing "Reception", "Doctor", and "Inventory" views are only accessible AFTER login.

---

## Phase 6: The Appointment Engine

**Goal:** Enable Receptionists to schedule patients with doctors. **Syllabus Target:** Working with Files (New JSON), Dictionary Manipulations.

### Sub-Phase 6.1: Appointment Data Structure

- **Focus:** Defining what an appointment looks like.
- **Concepts:** Data Modelling.

### Prompt for Antigravity (Copy & Paste):

Plaintext

We need to handle Appointments.

Update ``models.py`` and ``data_manager.py``.

1. In ``models.py``, create a class ``Appointment``.

- Attributes: ``appointment_id``, ``patient_id``, ``doctor_id``, ``date_time``, ``status`` (default "Scheduled").
- Methods: ``to_dict()``, and a static ``from_dict()``.

2. In ``data_manager.py``:
- Add ``APPOINTMENT_FILE = os.path.join(DATA_DIR, "appointments.json")``.
  - Update ``ensure_data_dir()`` to create this file if it doesn't exist.

Constraint: Simple, standard class structure.

### Sub-Phase 6.2: Scheduling UI (Receptionist View)

- **Focus:** Connecting Patients to Doctors.
- **Concepts:** UI Logic, Filtering Data.

#### Prompt for Antigravity (Copy & Paste):

Plaintext

Update ``main.py`` to add Scheduling features for the Receptionist.

In the "Reception Desk" section:

1. Add a new tab or expander: "Schedule Appointment".
2. Dropdown 1: Select Patient (from ``patients.json``).
3. Dropdown 2: Select Doctor (filter ``staff.json`` where ``role == "Doctor"``).
4. Input: Date & Time Picker (or text input for POC simplicity).
5. Button: "Book Appointment".
  - Logic: Create an ``Appointment`` object.
  - Save it to ``appointments.json``.
  - Show success message: "Appointment booked for [Patient] with [Doctor] on [Date]".

Constraint: Ensure you filter the staff list so only Doctors appear in the dropdown.

---

## Phase 7: The Clinical Loop (Refining Doctor's Cabin)

**Goal:** Make the doctor's workflow real (seeing assigned patients & saving history). **Syllabus**

**Target:** Crucial "Update" Logic (CRUD).

### Sub-Phase 7.1: The Doctor's "My Patients" View

- **Focus:** Filtering appointments for the logged-in doctor.
- **Concepts:** List Comprehensions, Logic.

#### Prompt for Antigravity (Copy & Paste):

Plaintext

Update the "Doctor's Cabin" in ``main.py`` to be smarter.

Instead of showing ALL patients:

1. Get the ``current_doctor_id`` from ``st.session_state['user']['pid']``.
2. Load ``appointments.json``.
3. Filter the list: Find appointments where ``doctor_id`` matches the current doctor AND ``status`` is "Scheduled".
4. Show this list in a ``st.selectbox`` ("My Appointments").
5. When an appointment is selected, automatically load that specific Patient's data for the Assessment view.

Constraint: This simulates a real hospital workflow where doctors only see their own patients.

### Sub-Phase 7.2: Fixing the "Save" Logic (Critical)

- **Focus:** Replacing the mock comment with actual persistence.
- **Concepts:** File I/O, Data Integrity.

#### Prompt for Antigravity (Copy & Paste):

Plaintext

Finally, fix the "Finalize Treatment" logic in ``main.py``.

In the Doctor's Cabin:

1. Look at the "Finalize & Save" button.
2. Currently, it assumes mock logic. Replace it with real logic:
  - Get the ``final_prescription`` text from the text area.
  - Find the actual ``Patient`` object in the loaded list.
  - Call ``patient.add_history()`` with the diagnosis and the *\*final edited\** prescription.
  - Call ``save_json(PATIENT_FILE, patients_data)`` to write changes to disk.
  - Optionally: Update the Appointment status to "Completed" in ``appointments.json``.

Constraint: This completes the CRUD cycle. Data must persist after this button is clicked.

## Phase 5: The Gatekeeper (Authentication & Admin Panel)

**Goal:** Implement the Login Screen and give Admins the power to create users (Staff/Doctors) with unique emails.

### Sub-Phase 5.1: Update Models for Security

**Prompt:**

Plaintext

I need to update `models.py` to support Authentication using Email and Password.

1. Modify the `Staff` class (and by extension `Doctor`):
  - Add `email` and `password` attributes to the `\_\_init\_\_` method.
  - Update `to\_dict` and the `\_\_init\_\_` call in `Doctor` to pass these new fields.
  - Add a method `verify\_password(self, input\_password)` that returns True if they match.
2. Constraint: Ensure `email` is treated as a unique identifier concept in your logic (we will enforce uniqueness in the Admin panel later).

### Sub-Phase 5.2: The Auth Manager

#### Prompt:

Plaintext

Create a new file `auth\_manager.py` to handle login logic.

1. Define a constant `ADMIN\_USER = {"email": "admin@hospital.com", "password": "admin123", "role": "Admin", "name": "System Admin"}`.
2. Create a function `login(email, password)`:
  - First, check if inputs match `ADMIN\_USER`. If yes, return the admin dict.
  - If not, load `data/staff.json`.
  - Loop through the staff list. If `user['email'] == email` AND `user['password'] == password`, return that user object.
  - Return `None` if authentication fails.

### Sub-Phase 5.3: The Admin Dashboard (Staff CRUD)

#### Prompt:

Plaintext

We need an "Admin Dashboard" in `main.py` that is only visible to the Admin.

1. Create a function or section `render\_admin\_panel()`:
  - **Create User Form:** Inputs for Name, Age, Contact, Role (Select: Doctor, Nurse, Staff), Email, Password.
  - **Doctor Specifics:** If "Doctor" is selected, show extra fields: Specialization, Shift Timing.
  - **Uniqueness Check:** Before saving, check `data/staff.json` to ensure the `email` does not already exist. If it does, show `st.error`.
  - **Save:** Create the appropriate object (`Staff` or `Doctor`), save to JSON.
2. **View Users:** Display a dataframe of all current staff (hide passwords). Add a "Delete" button for each row.

## Sub-Phase 5.4: The Login Wall (Refactoring Main)

### Prompt:

Plaintext

Refactor `main.py` to enforce a "Login First" policy.

1. **Session State:** Initialize `st.session_state['user']` to `None` if not present.
  2. **Login Screen:**
    - If `st.session_state['user']` is `None`:
      - Show a centered Login Form (Email & Password).
      - On Submit: Call `auth_manager.login()`.
      - If success: Set `st.session_state['user']` to the result and `st.rerun()`.
    - **Else (User is Logged In):**
      - Show the Sidebar and the App Content.
      - Add a "Logout" button in the sidebar that resets the session.
  3. **Role-Based Sidebar:**
    - If `Role == 'Admin'`: Show only "Admin Dashboard".
    - If `Role == 'Doctor'`: Show only "Doctor's Cabin" (Appointments & Treatment).
    - If `Role == 'Staff' or 'Nurse'`: Show "Reception Desk" (Patient Mgmt) and "Blood Bank".
- 

## Phase 6: Staff Workflows (Patients, Blood, & Scheduling)

**Goal:** Empower Staff to manage patients, inventory, and *book* appointments for doctors.

### Sub-Phase 6.1: Appointment Model

#### Prompt:

Plaintext

Update `models.py` and `data_manager.py` for Appointments.

1. Create class `Appointment`:
  - Attributes: `appointment_id`, `patient_id`, `doctor_id`, `patient_name`, `doctor_name`, `time_slot`, `status` (default "Scheduled").
  - Methods: `to_dict`, `from_dict`.
2. Update `data_manager.py` to handle `data/appointments.json`.

### Sub-Phase 6.2: Staff Dashboard (Reception + Booking)

### **Prompt:**

Plaintext

Update the "Reception Desk" view in `main.py` (visible to Staff/Nurses).

1. **\*\*Patient Management:\*\*** Keep the existing Registration and List view.
  2. **\*\*Book Appointment Section:\*\***
    - Select Patient (Dropdown from `patients.json`).
    - Select Doctor (Dropdown filtering `staff.json` where role='Doctor').
    - Input: Date/Time.
    - Action: Create `Appointment` object and save to `data/appointments.json`.
  3. **\*\*Blood Bank:\*\*** Keep the existing Blood Inventory view accessible to Staff.
- 

## **Phase 7: Doctor Workflows (Clinical Loop)**

**Goal:** Doctors see *their* schedule and treat *their* patients using AI.

### **Sub-Phase 7.1: My Schedule**

#### **Prompt:**

Plaintext

Update the "Doctor's Cabin" view in `main.py`.

1. **\*\*My Appointments:\*\***
  - Get current user's ID from `st.session\_state['user']['pid']`.
  - Load `appointments.json` and filter for this doctor.
  - Display a list/table of "Upcoming Appointments".
2. **\*\*Consultation Mode:\*\***
  - When a doctor selects an appointment from the list, open the "Assessment" view for that specific patient.

### **Sub-Phase 7.2: AI Treatment & History (The Fix)**

#### **Prompt:**

Plaintext

Refine the AI Consultation in "Doctor's Cabin".

1. **\*\*Context:\*\*** Pass the patient's *entire* medical history to the AI engine (not just current symptoms).

## 2. **\*\*Finalize Treatment:\*\***

- When the doctor clicks "Finalize & Save":
- Save the diagnosis/prescription to `patients.json` (append to history).
- Update the appointment status in `appointments.json` to "Completed".
- Show success message and return to the Appointment list.

## **Phase 7: Doctor's Clinical Loop**

### **Sub-Phase 7.1: The "My Appointments" Filter**

**Goal:** Replace the "All Patients" dropdown with a "My Scheduled Patients" dropdown.

**Prerequisite:** Ensure Phase 5 (Auth) is done so we have `st.session_state['user']`.

#### **Prompt for Antigravity:**

Plaintext

I need to update the "Doctor's Cabin" in `main.py` to filter patients.

Current Behavior: It shows ALL patients in the dropdown.

New Behavior: It should only show patients who have a "Scheduled" appointment with the currently logged-in Doctor.

Task:

1. **\*\*Get Current Doctor:\*\***
  - Retrieve `doctor_id = st.session_state['user']['pid']`.
2. **\*\*Load Appointments:\*\***
  - Load `data/appointments.json``.
  - Filter this list to find records where `doctor_id`` matches the current doctor AND `status` == "Scheduled"`.
  - Extract the list of `patient_id`s` from these appointments.
3. **\*\*Filter Patients:\*\***
  - Load `data/patients.json``.
  - Create a filtered list of Patient objects whose `pid`` is in the list of `patient_id`s` we just found.
4. **\*\*Update Dropdown:\*\***
  - Populate the `selectbox`` ONLY with this filtered list of patients.
  - If the list is empty, show `st.info("No appointments scheduled for today.")``.

### **Sub-Phase 7.2: Viewing Patient History**

**Goal:** Display the patient's past medical records *before* the doctor starts the new assessment.

## Prompt for Antigravity:

Plaintext

Refine the "Assessment" section in "Doctor's Cabin" to show Patient History.

Task:

### 1. **\*\*Display History:\*\***

- When a patient is selected, creating a new `st.expander("📁 View Patient History", expanded=True)`.
- Inside this expander, check `selected_patient['medical_history']`.
- If it exists, iterate through the list and display each entry (Date, Disease, AI Prediction) using `st.markdown` or a small table.
- If empty, show "No previous medical history."

### 2. **\*\*Context for AI:\*\***

- When calling `ai.predict_treatment(symptoms, history)`, ensure you are passing the *entire* `medical_history` list (as a string) into the `history` argument.
- This allows the AI to see past diseases when making a new diagnosis.

## Sub-Phase 7.3: Saving the Diagnosis (Closing the Loop)

**Goal:** Ensure the new treatment is saved to the history and the appointment is marked complete.

## Prompt for Antigravity:

Plaintext

Fix the "Finalize Treatment" button logic in "Doctor's Cabin".

Current Behavior: It prints a success message but saves nothing.

New Behavior: It must strictly save data to JSON.

Task:

### 1. **\*\*Prepare Record:\*\***

- Create a dictionary `new_record`:
  - `date`: Current Date (YYYY-MM-DD).
  - `diagnosis`: From AI result (or Doctor's edit).
  - `treatment`: From the "Final Prescription" text box.
  - `doctor_id`: Current logged-in ID.

### 2. **\*\*Update Patient:\*\***

- Load `patients.json`.
- Find the specific patient object.
- Append `new_record` to their `medical_history` list.



- Save `patients.json`.

3. **\*\*Close Appointment:\*\***

- Load `appointments.json`.
- Find the appointment for this patient/doctor.
- Change `status` from "Scheduled" to "Completed".
- Save `appointments.json`.

4. **\*\*Refresh:\*\***

- Show `st.success("Treatment Saved!")`.
- Call `st.rerun()` (This will remove the patient from the "Scheduled" list automatically).

**Goal:** Remove the redundant "Hospital System" header.

**Prompt for Antigravity:**

Plaintext

I need to fix a UI glitch in `main.py`.

Issue: The text "Hospital System" appears twice at the top of the sidebar.

Task:

1. Check `main.py` for `st.sidebar.title("🏥 Hospital System")`.
2. If it is called inside the `if user:` block AND also outside it, remove the one inside the loop.
3. Ensure `st.sidebar.title` is called exactly ONCE, preferably right after the user logs in and the sidebar renders.

**Step 2: Relocate Patient History**

**Goal:** Move the Patient History viewer to sit immediately below the Patient Selection dropdown, so the doctor sees the history *before* starting the assessment.

**Prompt for Antigravity:**

Plaintext

Refactor the layout of "Doctor's Cabin" in `main.py`.

Current Layout:

1. Select Patient Dropdown
2. Two Columns (Assessment | Treatment)
3. Patient History is hidden inside the Assessment column.

New Layout Requirement:

1. **\*\*Select Patient Dropdown:\*\*** Keep this at the top.
2. **\*\*Patient History Section:\*\***
  - Immediately BELOW the dropdown (before the columns start).
  - Create an expander: ``with st.expander(f"📁 Patient History: {selected_patient_name}", expanded=False):``
  - Inside, show the history table/list for the selected patient.
3. **\*\*Consultation Area:\*\***
  - BELOW the history section, start the ``col1, col2 = st.columns(2)`` layout for "Assessment" and "Treatment".

Constraint: This ensures the doctor can review past records easily before typing new symptoms.

**Goal:** Create reusable functions to check data format. **Syllabus Target:** Functions, Modules, Regular Expressions (optional but recommended, or string methods).

### Prompt for Antigravity (Copy & Paste):

Plaintext

I need to add robust input validation to my Hospital System.

Task: Create a new file named ``utils.py``.

1. **\*\*Define ``validate_contact(contact)``:**
  - Input: A string.
  - Logic:
    - Check if the string consists ONLY of digits (``contact.isdigit()``).
    - Check if the length is exactly 10.
  - Output: Return ``True`` if valid, ``False`` otherwise.
2. **\*\*Define ``validate_email(email)``:**
  - Input: A string.
  - Logic:
    - Check if "@" is present in the string.
    - Check if "." is present in the string.
  - Output: Return ``True`` if valid, ``False`` otherwise.

Constraint: Keep it simple and strictly following the requirements (10 digits, @ and .).

### Sub-Phase 10.2: Enforcing Rules in Models

**Goal:** Prevent invalid objects from even being created. **Syllabus Target:** Object Oriented Programming (Encapsulation/Validation in Constructor).

### Prompt for Antigravity (Copy & Paste):

Plaintext

Update `models.py` to enforce validation rules.

1. **Imports:** Import `validate\_contact` and `validate\_email` from `utils`.
2. **Update `Person.\_\_init\_\_`:**
  - Before setting `self.contact`, call `validate\_contact(contact)`.
  - If it returns `False`, raise a `ValueError("Contact must be a 10-digit number")`.
3. **Update `Staff.\_\_init\_\_` (and Doctor):**
  - If you have added the `email` field (from Phase 5), validation is needed there.
  - Before setting `self.email`, call `validate\_email(email)`.
  - If it returns `False`, raise a `ValueError("Invalid Email Format")`.

Constraint: This ensures that no bad data ever gets saved to our JSON files.

### Sub-Phase 10.3: UI Feedback (The User Experience)

**Goal:** Show friendly error messages in Streamlit instead of crashing the app. **Syllabus Target:** Streamlit UI (Conditional Rendering, Error Messages).

### Prompt for Antigravity (Copy & Paste):

Plaintext

Update `main.py` to provide feedback on validation errors.

1. **Imports:** Import `validate\_contact` and `validate\_email` from `utils`.
2. **Patient Registration (Reception Desk):**
  - Inside the "Register Patient" button logic:
  - Check `if not validate\_contact(contact):`.
  - If invalid: `st.error("Invalid Contact! Must be 10 digits.")` and stop execution.
  - Else: Proceed with creating the Patient object.
3. **Admin Dashboard (Add Staff):**
  - Inside the "Create User" button logic:
  - Check `if not validate\_email(email):`.
  - If invalid: `st.error("Invalid Email! Must contain '@' and '.")`.
  - Check `if not validate\_contact(contact):`.
  - If invalid: `st.error("Invalid Contact! Must be 10 digits.")`.
  - Else: Proceed with creating the Staff/Doctor object.

Constraint: Perform these checks BEFORE calling the class constructors to avoid handling exceptions in the UI code

## Phase 11: Patient Status Management (Fixing "Always Pending")

**Goal:** Add a section in the Reception Desk to Admit or Discharge patients.

### Prompt for Antigravity (Copy & Paste):

Plaintext

I need to fix the "Always Pending" issue in the Reception Desk.

Task: Update the "Reception Desk" section in `main.py`.

1. **\*\*Add a New Section:\*\***
  - Below the "Current Patients" table, add a divider `st.divider()` and a subheader `st.subheader("Manage Patient Status")`.
2. **\*\*Create the UI:\*\***
  - Create 3 columns: `c1, c2, c3 = st.columns(3)`.
  - **\*\*Column 1 (Select Patient):\*\*** Create a selectbox listing all patient names (format: "Name (ID)").
  - **\*\*Column 2 (New Status):\*\*** Create a selectbox with options `["ADMITTED", "DISCHARGED", "PENDING"]`.
  - **\*\*Column 3 (Action):\*\*** Add a button "Update Status".
3. **\*\*Implement Logic:\*\***
  - When the button is clicked:
    1. Load `patients.json`.
    2. Find the patient matching the selected ID.
    3. Update their `current_status` field to the new value.
    4. Save the list back to `patients.json` using `save_json`.
    5. Show `st.success` and call `st.rerun()` to refresh the table immediately.

Constraint: This allows the staff to actually move patients through the hospital flow.

## Phase 12: Email Alert System (Local SMTP)

**Goal:** Send email notifications for low stock using a local debug server. **Syllabus Target:** Modules, Working with External Libraries (`smtplib`), Networking Basics.

## Sub-Phase 12.1: The Email Service Module

**Goal:** Create the logic to format and send the email.

### Prompt for Antigravity (Copy & Paste):

Plaintext

I need to implement an Email Service for my Hospital System.

Create a new file `email\_service.py` inside the `utils/` folder (create the folder if needed).

Task:

1. **Imports:** Import `smtplib` and `email.message`.
  2. **Configuration:** Define constants:
    - `SMTP\_SERVER = "localhost"`
    - `SMTP\_PORT = 1025` (Standard debug port)
    - `SENDER\_EMAIL = "system@hospital.com"`
  3. **Function `send\_alert(blood\_group, quantity, recipient\_email)`:**
    - Create an `EmailMessage` object.
    - Set Subject: "URGENT: Low Stock Alert - [Blood Group]"
    - Set From: `SENDER\_EMAIL`
    - Set To: `recipient\_email`
    - Set Content: "Warning! The stock for {blood\_group} has dropped to {quantity} units. Please contact donors immediately."
- **Sending Logic:**
- Use `with smtplib.SMTP(SMTP\_SERVER, SMTP\_PORT) as server:`
  - Call `server.send\_message(msg)`.
- **Error Handling:** Wrap in a try-except block. Return `True` if sent, `False` if failed (and print the error).

## Sub-Phase 12.2: The Mock Server (Run in Terminal)

**Goal:** A simple command to receive emails. You don't need to write code for this; just run it.

**Instruction for You (Not a prompt for AI):** Open a **NEW Terminal** window (keep your Streamlit app running in the first one) and run this Python command. It starts a fake email server that listens on port 1025 and prints emails to the screen:

Bash

```
python -m smtpd -c DebuggingServer -n localhost:1025
```

(Note: If you are on Python 3.12+, this command might be deprecated. If so, let me know, and I will give you a tiny `mock_server.py` script to run instead.)

### Sub-Phase 12.3: Frontend Integration (The Button)

**Goal:** Add the "Send Alert" button to the Blood Bank UI.

#### Prompt for Antigravity (Copy & Paste):

Plaintext

Update the "Blood Bank" section in `main.py` to include the Email Alert feature.

1. **Imports:** Import `send_alert` from `utils.email_service`.
2. **Logic (Inside the "Blood Bank" block):**
  - Look at the `low_stock` logic (where you check `inventory.get_low_stock()`).
  - If `low_stock` is NOT empty:
    - Display the error message (as you currently do).
    - **New Addition:** Add a button: `if st.button("🚨 Send Email Alert to Admin"):`
    - **On Click:**
      - Loop through each item in `low_stock`.
      - Call `send_alert(item, current_quantity, "admin@hospital.com")`.
      - If successful, show `st.success(f"Alert sent for {item}")`.
      - If failed, show `st.error("Failed to connect to SMTP Server. Is it running?")`.

Constraint: This allows sending alerts with a single click as requested.

## Phase 13: Admin-Centric Donation Broadcast

### Sub-Phase 13.1: Cleaning the Blood Bank UI

**Goal:** Remove the button from the Staff view (Visual warning only).

#### Prompt for Antigravity (Copy & Paste):

Plaintext

I need to refactor the "Blood Bank" view in `main.py`.

Task:

1. Locate the `elif menu == "Blood Bank":` block.
2. Find the logic where `low_stock` is checked.
3. **Keep** the `st.error(f"🚨 Low Stock Alert: ...")` message (Visual warning is still needed).
4. **Remove** the entire `if st.button("🚨 Send Email Alert..."):` block.

Constraint: Staff should see the warning, but they cannot send emails anymore.

### **Sub-Phase 13.2: The Professional Donation Email**

**Goal:** Create a specific, well-written email template for patients.

#### **Prompt for Antigravity (Copy & Paste):**

Plaintext

Update `utils/email\_service.py` to handle mass broadcasts.

Task: Add a new function `send\_donation\_request(recipient\_email, patient\_name, missing\_blood\_type)`.

Logic:

1. **Subject:** `f"Urgent Appeal: {missing\_blood\_type} Blood Needed - Save a Life Today"`
2. **Body:** Use this professional template:

"""

Dear {patient\_name},

We hope you are in good health.

This is an urgent appeal from City Hospital. We are currently facing a critical shortage of **{missing\_blood\_type}** blood in our inventory.

As a registered member of our hospital network, we are reaching out to ask for your support. If you or anyone you know is eligible to donate, please visit our blood bank at your earliest convenience. Your donation could save a life today.

Location: City Hospital, Block A.

Hours: 9 AM - 8 PM.

Thank you for your kindness and support.

Sincerely,

Hospital Administration

"""

3. **Sending:** Use the existing SMTP logic (port 1025) to send this message.

### **Sub-Phase 13.3: Admin Dashboard Broadcast Logic**

**Goal:** Give the Admin the button to blast emails to all patients.

## Prompt for Antigravity (Copy & Paste):

### Plaintext

Update the "Admin Dashboard" in `main.py` to include the Alert System.

### Task:

1. **Imports:** Import `BloodInventory` from `models` and `send\_donation\_request` from `utils.email\_service`.
2. **New Section:** Inside the Admin view, add `st.divider()` and `st.subheader("🚑 Blood Stock Alerts")`.
3. **Logic:**
  - Instantiate `inventory = BloodInventory()`.
  - Get `low\_stock\_items = inventory.get\_low\_stock()`.
4. **UI:**
  - If `low\_stock\_items` is empty: Show `st.success("Blood stock levels are adequate.")`.
  - If `low\_stock\_items` HAS items:
    - Show `st.error(f"CRITICAL SHORTAGE DETECTED: {' '.join(low\_stock\_items)}")`.
    - Add a button: `if st.button("🚀 Broadcast Donation Request to All Patients")`:
5. **Broadcast Action (On Click):**
  - Load `patients.json`.
  - Create a progress bar: `progress = st.progress(0)`.
  - Loop through all patients:
    - For each `blood\_type` in `low\_stock\_items`:
      - Call `send\_donation\_request(patient.email, patient.name, blood\_type)`.
  - Update progress bar.
  - Show `st.success("Broadcast sent to all registered patients.")`.

Constraint: This sends one email per patient per missing blood type.



Tab 3

when the application is being loaded the first thing asked should log in based on their roles  
admin doctor and staff

admin can do CRUD for staff and doctors mail and password can be assigned by the admin  
which can be used them at the time logging in (email should be unique)

staff can manage patients and bloodbank

doctor can manage appointment and treatments

these are the remaining features

outline the major phases and prompts for the sub phases